

Proyecto LoyalKicks 2.0

Integrantes:

Mateo Debrott

Matias Herrera

DESARROLLO FULLSTACK I_004D

Profesora:

MARIANGELES ROBINSON ALHUCEMA

Introducción

En la actualidad, las empresas dedicadas al comercio de productos requieren sistemas informáticos eficientes que les permitan gestionar sus operaciones de manera rápida, segura y escalable. En este contexto, las arquitecturas basadas en microservicios se han convertido en una solución ampliamente utilizada, ya que permiten dividir una aplicación en servicios independientes que facilitan el mantenimiento, la escalabilidad y el desarrollo colaborativo.

El presente proyecto consiste en el desarrollo de LoyalKicks 2.0, una plataforma backend orientada a la gestión de una tienda de zapatillas. La solución fue implementada utilizando una arquitectura de microservicios, donde cada servicio es responsable de una funcionalidad específica del negocio, como la administración de clientes, productos, inventario, pedidos, pagos y otros procesos relacionados. Cada microservicio cuenta con su propia base de datos y se comunica mediante un API Gateway, permitiendo una integración centralizada y eficiente.

Además de la implementación de las operaciones CRUD necesarias para la gestión de la información, el proyecto incorpora pruebas unitarias e integración utilizando JUnit y Mockito, con el objetivo de garantizar la calidad, consistencia y correcto funcionamiento de los servicios desarrollados. Finalmente, se documentó técnicamente la solución y se preparó para su despliegue en un entorno de ejecución, siguiendo las buenas prácticas de desarrollo de software y arquitectura distribuida.

Este informe presenta el diseño, desarrollo, pruebas y resultados obtenidos durante la construcción de la solución, evidenciando la aplicación de los conocimientos adquiridos en el ámbito de los microservicios y el desarrollo backend moderno.

Objetivos

Objetivo General:

Desarrollar una solución backend para la gestión de una tienda de zapatillas mediante una arquitectura basada en microservicios, implementando operaciones CRUD, pruebas de software, documentación técnica y mecanismos de comunicación centralizada entre servicios para garantizar un sistema escalable, mantenible y eficiente.

Objetivos Específicos:

Diseñar e implementar los microservicios necesarios para gestionar los procesos principales de la tienda, tales como clientes, productos, inventario, pedidos, pagos y envíos.

Desarrollar operaciones CRUD completas en cada microservicio, permitiendo la administración eficiente de la información del sistema.

Implementar bases de datos independientes para cada microservicio, respetando los principios de una arquitectura distribuida.

Configurar un API Gateway para centralizar y gestionar la comunicación entre los distintos microservicios.

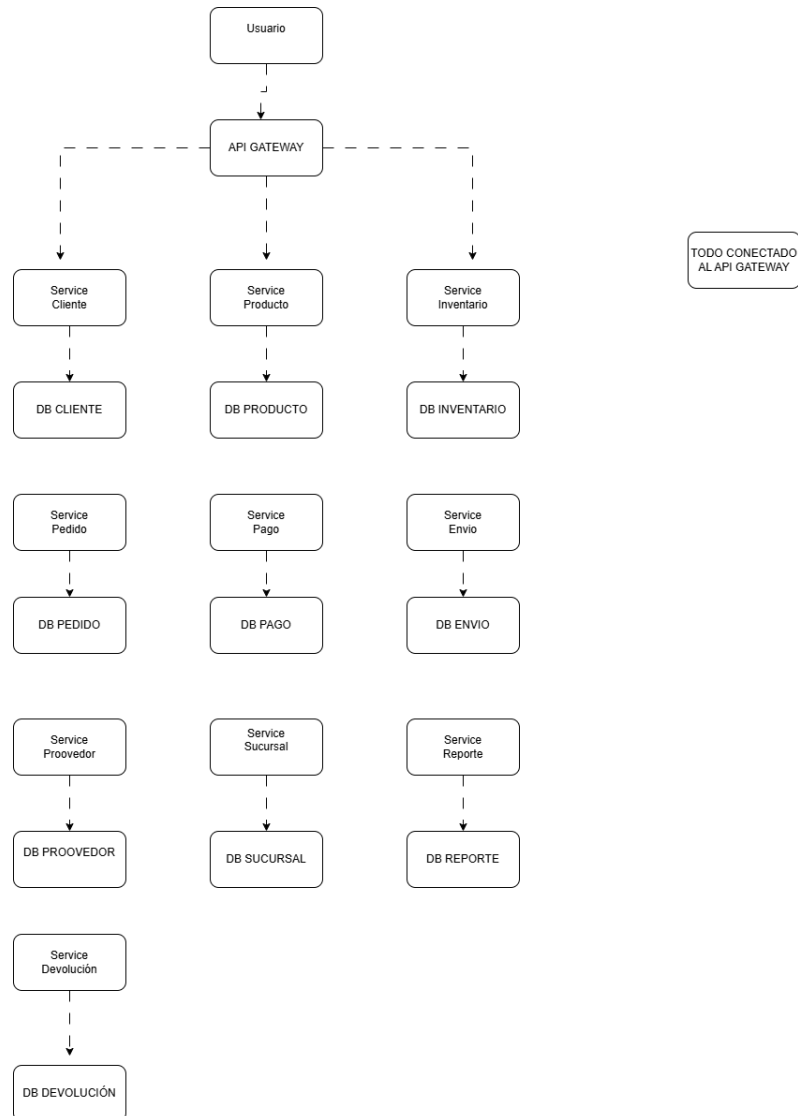
Realizar pruebas unitarias utilizando JUnit y Mockito para validar el correcto funcionamiento de la lógica de negocio.

Ejecutar pruebas de integración para verificar la comunicación y consistencia entre los distintos componentes del sistema.

Documentar técnicamente la arquitectura, endpoints, bases de datos y funcionalidades desarrolladas.

Desplegar la solución en un entorno local o remoto para comprobar su funcionamiento y accesibilidad.

Arquitectura lógica basada en microservicios de LoyalKicks 2.0



La solución LoyalKicks 2.0 fue desarrollada bajo una arquitectura de microservicios, donde cada componente es independiente y posee su propia base de datos. La comunicación entre los clientes y los servicios se realiza mediante un API Gateway, el cual actúa como punto único de entrada y distribuye las solicitudes hacia el microservicio correspondiente. Los microservicios implementados son Cliente, Producto, Inventario, Pedido, Pago, Envío, Proveedor, Sucursal, Devolución y Reporte. Esta arquitectura permite mejorar la escalabilidad, disponibilidad y mantenibilidad del sistema, ya que cada servicio puede desarrollarse, desplegarse y actualizarse de forma independiente sin afectar el funcionamiento de los demás componentes.

Tecnologías Utilizadas

Para el desarrollo del proyecto LoyalKicks 2.0 utilizamos diversas tecnologías y herramientas que permitieron implementar una arquitectura basada en microservicios, garantizando un sistema modular, escalable y fácil de mantener. A continuación, se describen las principales tecnologías empleadas:

Java 17: Lenguaje de programación utilizado para el desarrollo de los microservicios.

Spring Boot: Framework utilizado para la creación rápida de aplicaciones backend y microservicios.

Spring Data JPA: Herramienta que facilita la interacción con las bases de datos mediante el uso de repositorios y entidades.

MySQL: Sistema de gestión de bases de datos relacional utilizado para almacenar la información de cada microservicio.

Maven: Herramienta para la gestión de dependencias y construcción de proyectos Java.

Spring Cloud Gateway: Implementación del API Gateway encargada de centralizar y enrutar las solicitudes hacia los distintos microservicios.

Swagger/OpenAPI: Herramienta utilizada para documentar y probar los servicios REST desarrollados.

JUnit 5: Framework utilizado para la creación y ejecución de pruebas unitarias.

Mockito: Biblioteca empleada para simular dependencias durante las pruebas unitarias.

Git y GitHub: Herramientas utilizadas para el control de versiones y almacenamiento del código fuente.

Postman: Aplicación utilizada para probar y validar el funcionamiento de los endpoints REST.

Hibernate: Framework ORM encargado de mapear las entidades Java con las tablas de la base de datos.

Justificación de las Tecnologías

La combinación de Java 17 y Spring Boot permitió desarrollar una solución robusta y orientada a microservicios. Por su parte, MySQL proporcionó un almacenamiento confiable para cada servicio, mientras que Spring Cloud Gateway facilitó la comunicación centralizada entre los distintos componentes del sistema. Además, el uso de JUnit 5 y Mockito permitió verificar el correcto funcionamiento de la lógica de negocio mediante pruebas automatizadas, aumentando la calidad y confiabilidad de la aplicación.

Descripción de los Microservicios

La solución LoyalKicks 2.0 fue desarrollada bajo una arquitectura de microservicios, donde cada servicio es independiente y se encarga de una funcionalidad específica del negocio. Esta separación permite mejorar la escalabilidad, el mantenimiento y la disponibilidad del sistema. A continuación, se describen los microservicios implementados.

Cliente: Este microservicio administra la información de los clientes registrados en la plataforma.

Producto: Gestiona el catálogo de zapatillas y productos disponibles para la venta.

Inventario: Controla el stock disponible de los productos almacenados.

Pedido: Gestiona las compras realizadas por los clientes.

Pago: Administra la información relacionada con los pagos efectuados por los clientes.

Envío: Gestiona el proceso de despacho y entrega de productos.

Proovedor: Permite administrar la información de los proveedores de productos.

Sucursal: Administra las distintas sucursales o puntos de venta de la empresa.

Devolución: Gestiona las solicitudes de devolución realizadas por los clientes.

Reporte: Permite generar información y estadísticas para apoyar la toma de decisiones.

Gateway: El API Gateway actúa como el punto único de acceso al sistema y es responsable de dirigir las solicitudes hacia el microservicio correspondiente.

Base de Datos

Cliente: DB_CLIENTE

Producto: DB_PRODUCTO

Inventario: DB_INVENTARIO

Pedido: DB_PEDIDO

Pago: DB_PAGO

Envío: DB_ENVIO

Proovedor: DB_PROOVEDOR

Sucursal: DB_SUCURSAL

Devolución: DB_DEVOLUCION

Reporte: DB:REPORTE

Durante el desarrollo de esta etapa del proyecto se lograron implementar satisfactoriamente los componentes solicitados, permitiendo validar el correcto funcionamiento de la arquitectura basada en microservicios.

Los resultados obtenidos fueron los siguientes:

Funcionamiento de los CRUD

Se verificó el correcto funcionamiento de las operaciones de creación, consulta, actualización y eliminación (CRUD) en los distintos microservicios del sistema. Las pruebas realizadas demostraron que los datos son procesados y almacenados correctamente.

Integración entre Microservicios

La comunicación entre los microservicios se realizó de manera exitosa mediante solicitudes HTTP gestionadas a través del API Gateway. Esto permitió centralizar el acceso a los servicios y mejorar la organización de la arquitectura.

Pruebas Unitarias

Se implementaron pruebas unitarias utilizando JUnit y Mockito para validar la lógica de negocio de los servicios principales. Los resultados obtenidos permitieron detectar y corregir errores antes de la puesta en producción, aumentando la confiabilidad del sistema.

Pruebas de Integración

Las pruebas de integración verificaron el correcto funcionamiento de los controladores, servicios y repositorios, asegurando que los distintos componentes trabajaran de forma conjunta sin inconvenientes.

Despliegue Exitoso

El sistema fue desplegado en un entorno remoto, permitiendo acceder a los servicios desde fuera del entorno local. Las pruebas posteriores al despliegue confirmaron la estabilidad y disponibilidad de la aplicación.

Cumplimiento de Objetivos

Se cumplieron los objetivos planteados para esta etapa del proyecto, logrando:

Implementar una arquitectura basada en microservicios.

Centralizar las solicitudes mediante API Gateway.

Validar el funcionamiento con pruebas unitarias e integración.

Mantener la correcta ejecución de las operaciones CRUD.

Desplegar el sistema en un entorno accesible y funcional.

Los resultados obtenidos demuestran que la solución desarrollada cumple con los requisitos técnicos establecidos y proporciona una base sólida para futuras mejoras y ampliaciones del sistema.

Conclusión

El desarrollo de la presente etapa del proyecto permitió consolidar los conocimientos adquiridos durante el semestre en relación con el diseño e implementación de sistemas backend basados en arquitectura de microservicios. La aplicación de tecnologías como Spring Boot, Spring Cloud Gateway, JUnit y Mockito contribuyó a la construcción de una solución modular, escalable y alineada con las buenas prácticas del desarrollo de software.

Uno de los principales logros obtenidos fue la correcta implementación de los distintos microservicios que componen el sistema, permitiendo la gestión independiente de las funcionalidades relacionadas con productos, clientes, pedidos, pagos, inventario y otros módulos del negocio. Esta separación favorece el mantenimiento, la escalabilidad y la reutilización del código, aspectos fundamentales en el desarrollo de aplicaciones empresariales modernas.

Asimismo, la incorporación de un API Gateway permitió centralizar el acceso a los servicios, facilitando la comunicación entre componentes y mejorando la organización general de la arquitectura. Esta implementación contribuyó a una gestión más eficiente de las solicitudes y a una mejor experiencia para los consumidores de la API.

En conclusión, los objetivos planteados para esta etapa fueron cumplidos satisfactoriamente. El proyecto desarrollado constituye una solución funcional y robusta, capaz de servir como base para futuras mejoras y ampliaciones. Además, la experiencia obtenida durante su desarrollo fortaleció competencias técnicas esenciales en áreas como arquitectura de software, pruebas automatizadas, integración de servicios y despliegue de aplicaciones, contribuyendo significativamente a la formación profesional en el ámbito de la ingeniería informática.